Learning Objectives

Learners will be able to...

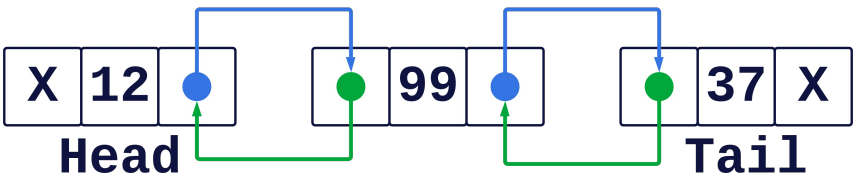
- **Define doubly linked list**
- **Differentiate a doubly linked list from a singly linked list**
- **Implement a doubly linked list**
- **Analyze the complexities of different linked list operations**

Doubly Linked Lists

What is a Doubly Linked List?

Now that we have a good understanding of a singly linked list, we can extend this concept and introduce a doubly linked list. A **doubly linked list (DLL)** is a list of nodes, where each node has two pointers. One references the next node in the list, while the other pointer references the previous node. This allows for bidirectional traversal, making some algorithms more efficient and easier to implement.

The image below is a representation of a doubly linked list. In the case of our head node, we can see the pointer for the previous node has the value `nullptr`, while the pointer for the next node references the next `Node` object. The tail node has a pointer that references the previous `Node` object, but the next pointer has the value of `nullptr`. Nodes between the head and tail have pointers that reference both the previous and next nodes.



Implementing a Doubly Linked List

To create a doubly linked list, we are going to make some slight changes to the code for a singly linked list. Starting with the `Node` class, add the attribute `prev` that is of type `Node*`. In the constructor, set the initial value of `prev` to `nullptr`.

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};
```

Next, create the `DoublyLinkedList` class. Just as before, this class will encapsulate all of the operations we want to perform on the doubly linked list. However, we are going to keep track of the tail node. This is not a requirement of a doubly linked list, but you should see how doing this adds some efficiency.

```
class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }
};
```

We also need to create the `printList` method so we can see the changes we make to our doubly linked list. The logic of this method does not change from a singly linked list to a doubly linked list. Create a `Node*` object set to the head. As long as this object is not `nullptr`, print the value stored in the node. Update the node to the `next` attribute.

```
    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }
```

▼ Code

Your code should look like this:

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }
};
```

Insertion

Adding Nodes

We will delve into the nitty-gritty details of each operation in a doubly linked list. The inclusion of a `prev` pointer in each node in a doubly linked list allows us to navigate the list in both directions, which can lead to some optimizations and ease of manipulation. Understanding these operations in detail provides you with the ability to manipulate doubly linked lists more efficiently. Let's start with insertion.

At the Beginning

Inserting a node at the beginning is straightforward. You allocate a new node, set its `next` pointer to the current head. If the doubly linked list already has a node in it, then you update the `prev` pointer of the old head to point to the new node. Update the `head` attribute to point to the new node. If our list only has one node (`head->next` is `nullptr`), then that node is both the head and the tail. In this case, update `tail` to point to the newly added node.

```
    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }
```

Create the `main` method that instantiates a `DoublyLinkedList` object and adds a node to the beginning. Print the list.

```
int main() {
    DoublyLinkedList list;
    list.insertAtBeginning(42);
    list.printList();
    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }
};

int main() {
    DoublyLinkedList list;
    list.insertAtBeginning(42);
    list.printList();
    return 0;
}
```

At the End

Because we are keeping track of the tail node, inserting at the end of the list is much more efficient. Start by creating a new `Node`. If the list is empty (`head` is equal to `nullptr`), then set `head` and `tail` to the new node. Otherwise, set the next attribute of the old `tail` to the new node. Then set the `prev` attribute of the new node to the old tail. Finally, set the tail attribute to the new `Node`.

```
    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
```

Update the `main` method to add a node at the end of the list. Just to verify that our logic is correct, call the `insertAtEnd` method before calling `insertAtBeginning`.

```
int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtEnd(3325);
    list.printList();

    std::cout << "After: ";
    list.insertAtBeginning(42);
    list.printList();
    return 0;
}
```

▼ **Code**

> Your code should look like this:
>
> ```
> class Node {
> public:
>     int data;
>     Node* next;
>     Node* prev;
>
>     Node(int data) {
> ```

```
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }

    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
};

int main() {
    DoublyLinkedList list;
```

```
    std::cout << "Before: ";
    list.insertAtEnd(3325);
    list.printList();

    std::cout << "After: ";
    list.insertAtBeginning(42);
    list.printList();
    return 0;
}
```

At a Specific Position

To insert at a specific position, traverse the list to that point, update the next and `prev` pointers for the node before and the node after, and then insert the new node in between.

The `insertAtPos` method should take data for the new node as well as the position of the insertion. First, check to see if the position is not valid; i.e., if it is less than `1`. Print a message to the user for an invalid position. Note, linked lists do not have indices so we are going to assume the first position is represented by the number `1`. If the position is `1`, then call the `insertAtBeginning` method and pass it the `data` variable.

```
    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {

        }
    }
```

Next, create the `current` variable which is a pointer to the current node in the list. We also need a new node that will be inserted. The variable `currentPos` is a counter variable to help us end the traversal. Use a while loop to traverse the list. Stop when `currentPos` is greater than or equal to the specified position minus `1` or when the value of current is `nullptr` (gone past the end of the list).

```
    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before the
        desired position.
            while (currentPos < pos - 1 && current != nullptr) {
                current = current->next;
                currentPos++;
            }

        }
    }
```

If we have traversed past the end of the list (`current` is equal to `nullptr`), print a message that the specified position is not valid.

```
    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before the
        desired position.
            while (currentPos < pos - 1 && current != nullptr) {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." << std::endl;
                return;
            }

        }
    }
```

Finally, we are ready to insert the new node. Set the `prev` attribute of the new node to the current node. Set the `next` attribute of the new node to the `next` attribute of the current node. If the current node is the `tail`, set the tail to the new node. Otherwise, set the `prev` attribute of the node past the current node to the new node. Lastly, set the `next` attribute of the current node to the new node.

```
    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before the
        desired position.
            while (currentPos < pos - 1 && current != nullptr) {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." << std::endl;
                return;
            }

            // Adjust links to insert the new node at the
        specified position.
            newNode->prev = current;
            newNode->next = current->next;
           if (current->next == nullptr) {
                tail = newNode;
            } else {
                current->next->prev = newNode;
            }
            current->next = newNode;
        }
    }
```

Update the `main` method so that our doubly linked list has four nodes in it. Then add a node at the third position.

```
int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.insertAtPos(5, 3);
    list.printList();
    return 0;
}
```

▼ **Code**

> Your code should look like this:

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;
```

```
    if (head != nullptr) {
        head->prev = newNode;
    }

    head = newNode;

    if (head->next == nullptr) {
        tail = newNode;
    }
}

void insertAtEnd(int data) {
    Node* newNode = new Node(data);

    if (head == nullptr) {
        head = newNode;
        tail = newNode;
    } else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
}

void insertAtPos(int data, int pos) {
    if (pos < 1) {
        std::cout << "Invalid position." << std::endl;
        return;
    }

    if (pos == 1) {
        insertAtBeginning(data);
    } else {
        Node* current = head;
        Node* newNode = new Node(data);
        int currentPos = 1;

        // Traverse the list to find the node before
    the desired position.
        while (currentPos < pos - 1 && current !=
    nullptr) {
            current = current->next;
            currentPos++;
        }

        if (current == nullptr) {
            std::cout << "Invalid position." <<
    std::endl;
            return;
        }

        // Adjust links to insert the new node at the
    specified position.
        newNode->prev = current;
        newNode->next = current->next;
       if (current->next == nullptr) {
            tail = newNode;
```

```
            } else {
                current->next->prev = newNode;
            }
            current->next = newNode;
        }
    }
};

int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.insertAtPos(5, 3);
    list.printList();
    return 0;
}
```

challenge

Try these variations:

- Test the `insertAtPos` method with the valid positions: 1, 2, 3, 4, and 5.
- Test the `insertAtPos` method with the invalid positions: 0 and 6.

Deletion

Removing Nodes

Just like the insertion operations, we are going to cover how to remove nodes from the beginning, end, and at a specific position in a list.

From the Beginning

Deleting from the beginning is straightforward. Make the head point to the second node (`head->next`) and update the `prev` pointer of the new head to `nullptr`.

```
    void deleteAtBeginning() {

        if (head == nullptr) {
            return;
        }

        head = head->next;
        if (head != nullptr) {
            head->prev = nullptr;
        }
    }
```

Update the `main` method so that the list contains the values `1`, `2`, `3`, and `4`. Then remove the first node from the list with the `deleteAtBeginning` method.

```
int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.deleteAtBeginning();
    list.printList();
    return 0;
}
```

▼ Code

> Your code should look like this:

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }

    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
```

```
            tail = newNode;
        }
    }

    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before
        the desired position.
            while (currentPos < pos - 1 && current !=
        nullptr) {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." <<
        std::endl;
                return;
            }

            // Adjust links to insert the new node at the
        specified position.
            newNode->prev = current;
            newNode->next = current->next;
           if (current->next == nullptr) {
                tail = newNode;
            } else {
                current->next->prev = newNode;
            }
            current->next = newNode;
        }
    }

    void deleteAtBeginning() {

        if (head == nullptr) {
            return;
        }

        head = head->next;
        if (head != nullptr) {
            head->prev = nullptr;
        }
    }
};

int main() {
```

```
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.deleteAtBeginning();
    list.printList();
    return 0;
}
```

From the End

Because we are keeping track of the tail node, you can quickly delete the last node. Update the `tail` to point to the second last node and update the `next` pointer of the new tail to `nullptr`.

```
    void deleteAtEnd() {
        tail = tail->prev;
        tail->next = nullptr;
    }
```

Update the `main` method so that the list contains the values `1`, `2`, `3`, and `4`. Then remove the tail node from the list with the `deleteAtEnd` method.

```
int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.deleteAtEnd();
    list.printList();
    return 0;
}
```

▼ Code

> Your code should look like this:

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }

    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
```

```
            tail = newNode;
        }
    }

    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before
        the desired position.
            while (currentPos < pos - 1 && current !=
        nullptr) {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." <<
        std::endl;
                return;
            }

            // Adjust links to insert the new node at the
        specified position.
            newNode->prev = current;
            newNode->next = current->next;
           if (current->next == nullptr) {
                tail = newNode;
            } else {
                current->next->prev = newNode;
            }
            current->next = newNode;
        }
    }

    void deleteAtBeginning() {

        if (head == nullptr) {
            return;
        }

        head = head->next;
        if (head != nullptr) {
            head->prev = nullptr;
        }
    }

    void deleteAtEnd() {
        tail = tail->prev;
```

```
        tail->next = nullptr;
    }
};

int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.deleteAtEnd();
    list.printList();
    return 0;
}
```

From a Specific Position

Deleting a node from a specific position is similar to insertion, however, there are a few important differences. Start by validating that the position to remove is greater than 1. If the node to remove is at position 1, call the `deleteAtBeginning` method. This traversal will stop at the node to be removed (`currentPos < pos`). Again, if you traverse past the end of the list, print a message to the user.

```
    void deleteAtPos(int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            deleteAtBeginning();
        } else {
            Node* current = head;
            int currentPos = 1;

            // Traverse the list to find the node at the desired
        position.
            while (currentPos < pos && current != nullptr) {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." << std::endl;
                return;
            }
        }
    }
```

When it comes to actually removing the node, set the `next` attribute of the node before the current node to `next` attribute of the current node. If the current node is the tail, set the tail to the `prev` attribute of the current node. Otherwise, set the `prev` attribute of the node after the current node to the node that comes before the current node.

```
    void deleteAtPos(int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            deleteAtBeginning();
        } else {
            Node* current = head;
            int currentPos = 1;

            // Traverse the list to find the node at the desired
        position.
            while (currentPos < pos && current != nullptr) {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." << std::endl;
                return;
            }

            // Adjust links to delete the node at the specified
        position.
            current->prev->next = current->next;
            if (current->next == nullptr) {
                tail = current->prev;
            } else {
                current->next->prev = current->prev;
            }
            delete current;
        }
    }
```

Update the `main` method so that the list contains the values `1`, `2`, `3`, and `4`. Then remove the second node from the list with the `deleteAtPos` method.

```
int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.deleteAtPos(2);
    list.printList();
    return 0;
}
```

▼ **Code**

> Your code should look like this:

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }

    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
```

```
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }

    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before
        the desired position.
            while (currentPos < pos - 1 && current !=
        nullptr) {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." <<
        std::endl;
                return;
            }

            // Adjust links to insert the new node at the
        specified position.
            newNode->prev = current;
            newNode->next = current->next;
           if (current->next == nullptr) {
                tail = newNode;
            } else {
                current->next->prev = newNode;
            }
            current->next = newNode;
        }
    }

    void deleteAtBeginning() {

        if (head == nullptr) {
            return;
        }

        head = head->next;
        if (head != nullptr) {
            head->prev = nullptr;
        }
    }
```

```
    void deleteAtEnd() {
        tail = tail->prev;
        tail->next = nullptr;
    }

    void deleteAtPos(int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            deleteAtBeginning();
        } else {
            Node* current = head;
            int currentPos = 1;

            // Traverse the list to find the node at the
        desired position.
            while (currentPos < pos && current != nullptr)
        {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." <<
        std::endl;
                return;
            }

            // Adjust links to delete the node at the
        specified position.
            current->prev->next = current->next;
            if (current->next == nullptr) {
                tail = current->prev;
            } else {
                current->next->prev = current->prev;
            }
            delete current;
        }
    }
};

int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    list.deleteAtPos(2);
    list.printList();
```

```
    return 0;
}
```

challenge

Try these variations:

- Test the `deleteAtPos` method with the valid positions: `1`, `2`, `3`, and `4`.
- Test the `deleteAtPos` method with the invalid positions: `0` and `5`.

Traversal & Searching

Navigating a List

Having pointers for both the previous and next nodes allows you to move around a doubly linked list with a bit more freedom.

Traversing

The pointer for the previous node (as well as knowing the tail) means you can traverse a list in reverse. Create the method `reversePrint` which will print out the values of the nodes in reverse order. Instantiate the variable `current` and set it to the tail. As long as the current node is not `nullptr`, print the value stored in the node and set `current` to the previous node.

```
    void reversePrint() {
        Node* current = tail;

        while (current != nullptr) {
            std::cout << current->data << " ";
            current = current->prev;
        }
        std::cout << std::endl;
    }
```

Update the `main` method to add four nodes to the list, and then call the `reversePrint` method.

```
int main() {
    DoublyLinkedList list;

    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);

    std::cout << "Normal: ";
    list.printList();

    std::cout << "Reverse: ";
    list.reversePrint();
    return 0;
}
```

▼ Code

> Your code should look like this:

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }

    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
```

```
    }
}

void insertAtPos(int data, int pos) {
    if (pos < 1) {
        std::cout << "Invalid position." << std::endl;
        return;
    }

    if (pos == 1) {
        insertAtBeginning(data);
    } else {
        Node* current = head;
        Node* newNode = new Node(data);
        int currentPos = 1;

        // Traverse the list to find the node before
    the desired position.
        while (currentPos < pos - 1 && current !=
    nullptr) {
            current = current->next;
            currentPos++;
        }

        if (current == nullptr) {
            std::cout << "Invalid position." <<
    std::endl;
            return;
        }

        // Adjust links to insert the new node at the
    specified position.
        newNode->prev = current;
        newNode->next = current->next;
       if (current->next == nullptr) {
            tail = newNode;
        } else {
            current->next->prev = newNode;
        }
        current->next = newNode;
    }
}

void deleteAtBeginning() {

    if (head == nullptr) {
        return;
    }

    head = head->next;
    if (head != nullptr) {
        head->prev = nullptr;
    }
}

void deleteAtEnd() {
    tail = tail->prev;
    tail->next = nullptr;
```

```
    }

    void deleteAtPos(int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            deleteAtBeginning();
        } else {
            Node* current = head;
            int currentPos = 1;

            // Traverse the list to find the node at the
        desired position.
            while (currentPos < pos && current != nullptr)
        {
                current = current->next;
                currentPos++;
            }

            if (current == nullptr) {
                std::cout << "Invalid position." <<
        std::endl;
                return;
            }

            // Adjust links to delete the node at the
        specified position.
            current->prev->next = current->next;
            if (current->next == nullptr) {
                tail = current->prev;
            } else {
                current->next->prev = current->prev;
            }
            delete current;
        }
    }

    void reversePrint() {
        Node* current = tail;

        while (current != nullptr) {
            std::cout << current->data << " ";
            current = current->prev;
        }
        std::cout << std::endl;
    }
};

int main() {
    DoublyLinkedList list;

    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
```

```
    std::cout << "Normal: ";
    list.printList();

    std::cout << "Reverse: ";
    list.reversePrint();
    return 0;
}
```

Searching

Searching a doubly linked list is still done in a linear manner. Let's take a look at how we can simplify some of our other operations using a search method.

Start by creating the `findNode` method. This method takes a value stored in a node as a parameter. Create the `current` variable and set its value to the head. Traverse through the list. If the data in the node matches the value passed to the method, return the node. If you finish traversing the list and have not found the value, return `nullptr` which signifies the node is not in the list.

```
Node* findNode(int value) {
    Node* current = head;

    while (current != nullptr) {
        if (current->data == value) {
            return current;
        }
        current = current->next;
    }

    return nullptr; // node not found
}
```

We can now create the `deleteNode` method that removes a given node from the list. Pass the method the node you want to delete. If either the list is empty (`head` is `nullptr`) or the node is not in the list (`del` is `nullptr`) then use the `return` statement to exit the method. If the list only has one node (`head == tail`), then set both the head and tail to `nullptr`. If you are deleting the head, then call `deleteAtBeginning`. If you are deleting the tail, call `deleteAtEnd`. Otherwise set the `next` attribute of the node before `del` to the node after `del`. Then set the `prev` attribute of the node after `del` to the node before `del`.

```
    void deleteNode(Node* del) {
        if (head == nullptr || del == nullptr) return;

        if (head == tail) {
            delete head;
            head = nullptr;
            tail = nullptr;
            return;
        }

        if (head == del) {
            deleteAtBeginning();
            return;
        }

        if (tail == del) {
            deleteAtEnd();
            return;
        }

        del->prev->next = del->next;
        del->next->prev = del->prev;
        delete del;
    }
```

Update the `main` method so there are four nodes in the list. Use the `findNode` method to locate the node with the value of `3`. Pass this node to the `deleteNode` method.

```
int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    Node* target = list.findNode(3);
    list.deleteNode(target);
    list.printList();
    return 0;
}
```

▼ Code

> Your code should look like this:
>
> ```
> class Node {
> public:
> ```

```
    int data;
    Node* next;
    Node* prev;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
        this->prev = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }

    void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        newNode->next = head;

        if (head != nullptr) {
            head->prev = newNode;
        }

        head = newNode;

        if (head->next == nullptr) {
            tail = newNode;
        }
    }

    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
```

```
void insertAtPos(int data, int pos) {
    if (pos < 1) {
        std::cout << "Invalid position." << std::endl;
        return;
    }

    if (pos == 1) {
        insertAtBeginning(data);
    } else {
        Node* current = head;
        Node* newNode = new Node(data);
        int currentPos = 1;

        // Traverse the list to find the node before
    the desired position.
        while (currentPos < pos - 1 && current !=
    nullptr) {
            current = current->next;
            currentPos++;
        }

        if (current == nullptr) {
            std::cout << "Invalid position." <<
    std::endl;
            return;
        }

        // Adjust links to insert the new node at the
    specified position.
        newNode->prev = current;
        newNode->next = current->next;
       if (current->next == nullptr) {
            tail = newNode;
        } else {
            current->next->prev = newNode;
        }
        current->next = newNode;
    }
}

void deleteAtBeginning() {

    if (head == nullptr) {
        return;
    }

    head = head->next;
    if (head != nullptr) {
        head->prev = nullptr;
    }
}

void deleteAtEnd() {
    tail = tail->prev;
    tail->next = nullptr;
}
```

```
void deleteAtPos(int pos) {
    if (pos < 1) {
        std::cout << "Invalid position." << std::endl;
        return;
    }

    if (pos == 1) {
        deleteAtBeginning();
    } else {
        Node* current = head;
        int currentPos = 1;

        // Traverse the list to find the node at the
    desired position.
        while (currentPos < pos && current != nullptr)
    {
            current = current->next;
            currentPos++;
        }

        if (current == nullptr) {
            std::cout << "Invalid position." <<
    std::endl;
            return;
        }

        // Adjust links to delete the node at the
    specified position.
        current->prev->next = current->next;
        if (current->next == nullptr) {
            tail = current->prev;
        } else {
            current->next->prev = current->prev;
        }
        delete current;
    }
}

void reversePrint() {
    Node* current = tail;

    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->prev;
    }
    std::cout << std::endl;
}

Node* findNode(int value) {
    Node* current = head;

    while (current != nullptr) {
        if (current->data == value) {
            return current;
        }
        current = current->next;
    }
```

```
        return nullptr; // node not found
    }


    void deleteNode(Node* del) {
        if (head == nullptr || del == nullptr) return;

        if (head == tail) {
            delete head;
            head = nullptr;
            tail = nullptr;
            return;
        }

        if (head == del) {
            deleteAtBeginning();
            return;
        }

        if (tail == del) {
            deleteAtEnd();
            return;
        }

        del->prev->next = del->next;
        del->next->prev = del->prev;
        delete del;
    }

};

int main() {
    DoublyLinkedList list;

    std::cout << "Before: ";
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    list.printList();

    std::cout << "After: ";
    Node* target = list.findNode(3);
    list.deleteNode(target);
    list.printList();
    return 0;
}
```

challenge

Try these variations:

- Test the `deleteNode` method with the valid values: `1`, `2`, `3`, and `4`.
- Test the `deleteNode` method with the invalid values: `0` and `5`.

Doubly Linked List Analysis

Time Complexity Analysis of Doubly Linked Lists Operations

Understanding the time complexity of various operations in data structures is crucial for writing efficient code. In this section, we will analyze the time complexity for the fundamental operations in a doubly linked list: insertion, deletion, traversal, and searching.

Insertion

Time complexities for insertion operations depend on where the node is being inserted and if the tail node is known.

- **At the Beginning** - Inserting a node at the beginning of a DLL takes constant time $O(1)$, as you're merely changing a few pointers and updating the head of the list.
- **At the End** - Inserting a node at the end can be $O(n)$ if you start from the head and traverse the list to find the last element. However, this operation can be optimized to $O(1)$ if a tail pointer is maintained.
- **At a Specific Position** - Inserting a node at a specific position would require traversing the list up to that point, making the time complexity $O(n)$.

Deletion

Just like insertion, the time complexity can vary based on the location of the node to be deleted and knowing the tail node.

- **At the Beginning** - Deleting at the beginning takes $O(1)$ time.
- **At the End** - Deleting at the end also takes $O(1)$ time if a `tail` pointer is maintained; otherwise, it takes $O(n)$.
- **At a Specific Position** - If the node to be deleted is known and given as a pointer, the operation can be done in $O(1)$. Otherwise, it's $O(n)$.

Traversal & Searching

- **Traversal** - Traversing the entire list to perhaps print out elements or perform any operation on them will take $O(n)$ time, as you have to visit each node once.
- **Searching** - Searching for an element would require traversing the list until you find the element. In the worst case, you'd need to go through

the entire list, yielding a time complexity of $O(n)$.

Table of Operations

The table below summarizes the time complexities for the various operations covered. This includes optimized and non-optimized doubly linked lists.

Operation	Time Complexity	Optimized Time Complexity
Insertion (Beginning)	$O(1)$	$O(1)$
Insertion (End)	$O(n)$	$O(1)$ (with tail pointer)
Insertion (Position)	$O(n)$	$O(n)$
Deletion (Beginning)	$O(1)$	$O(1)$
Deletion (End)	$O(n)$	$O(1)$ (with tail pointer)
Deletion (Position)	$O(n)$ or $O(1)$	$O(1)$ (if node is known)
Traversal	$O(n)$	$O(n)$
Searching	$O(n)$	$O(n)$

Formative Assessment 1

Formative Assessment 2